

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3.1.2

HOSTING ELSEWHERE

 Copy as Markdown

Hosting elsewhere

You can host Serverpod anywhere, running Dart directly or through a Docker container.

Required services

Serverpod needs a database with the correct tables added, unless the project was created without a database, for example with `serverpod create --no-interactive --no-database`. Serverpod can also optionally use Redis. You enable Redis in your configuration files.

Configuration files

Serverpod has three main configuration files, one for each mode the server can run in: `development`, `staging`, and `production`. The files are located in the `config/` directory.

Two options control how the server starts:

- `--mode` selects the configuration file. By default, the server starts in development mode.
- `--server-id` sets the id of each server when you run multiple servers in a cluster. By default, the server runs as id `default`.

For example, to start the server in production mode with id `2`, run the following command:

```
$ dart bin/main.dart --mode production --server-id 2
```

Running all servers with the same id can also be valid. This helps on platforms such as AWS Fargate, where individual server ids are hard to configure.

By default, Serverpod will listen on ports 8080, 8081, and 8082. The ports are used by the API server, Serverpod Insights, and the [web server](#). You can configure the ports in the configuration files. Most often, you



Ask AI

will want to place your server or servers behind a load balancer that handles the SSL certificates for your server and maps the traffic to different domain addresses and ports (typically 443 for HTTPS).

Server roles

Serverpod can assume different roles depending on your configuration. If you run Serverpod on a single server or a cluster of servers, you typically will want to use the default `monolith` role. If you use a serverless service, use the `serverless` role. When Serverpod runs as a monolith, it will handle all maintenance tasks, such as health checks and future calls. If you run it serverless, you will need to schedule a cron job to start Serverpod in the `maintenance` role once every minute if you need support for future calls and health checks.

Role	Function
<code>monolith</code>	Handles incoming connections and maintenance tasks. Allows the server to contain a state. Default role.
<code>serverless</code>	Only handles incoming connections.
<code>maintenance</code>	Runs the maintenance tasks once, then exits.

You can specify the role of your server when you launch it by setting the `--role` argument.

```
$ dart bin/main.dart --role serverless
```

For connection pools, Redis, isolates, and other production scale concerns beyond roles, see [Scalability](#).

Docker container

Running Serverpod through a Docker container is often the best option as it provides a well-defined environment. It's also easy to integrate into your build and deployment process and runs well on most platforms.

You will get a `Dockerfile` created in your server directory when you set up a new project. Build it from the project root, because it also copies the `pubspec.lock` file stored there:

```
$ docker build -f <project>_server/Dockerfile .
```

The file works out of the box but can be tailored to your needs. It has no build options, but you can define environment variables when running it. The following variables are supported.

Environment variable	Meaning
<code>runmode</code>	The run mode to start the server in, possible values are <code>development</code> , <code>staging</code> , or <code>production</code> (the <code>Dockerfile</code> default).
<code>serverid</code>	Identifier of your server, default is <code>default</code>
<code>logging</code>	Logging mode at startup, default is <code>normal</code> , but you can specify <code>verbose</code> to get more information during startup which can help with debugging.
<code>role</code>	The role that the server will assume, possible values are <code>monolith</code> (default), <code>serverless</code> , or <code>maintenance</code> .